Object Oriented Programming - Viva Questions

Course Code: CM31203 | Units 2 to 6

Unit 2 : Functions in C++

Q1. What is a function and why are functions used in C++?

Ans. A function is a named, self-contained block of code that performs a specific task. Functions give code modularity, reusability of logic, easier debugging and testing, and a better program structure.

Q2. What are the two types of functions in C++?

Ans. Library functions (built-in, such as printf() and sqrt()) and user-defined functions written by the programmer for a specific task.

Q3. Give the syntax of a user-defined function.

Ans. return_type function_name(parameter_list) { function body }

Q4. What is the role of main() in a C++ program?

Ans. It is the starting point of execution. Only one main() is allowed in a program. It can be void main() in Turbo C++ or int main() in standard C++.

Q5. What is function prototyping and why is it needed?

Ans. A prototype is a declaration that tells the compiler the function name, return type, and the number and type of arguments. It is placed before main() so the compiler knows the function signature before the call.

Q6. What is call by reference?

Ans. The memory address of the actual parameters is passed to the function, so any change made in the formal parameters is reflected in the actual ones. For example void swap(int &a, int &b).

Q7. What is return by reference and where is it useful?

Ans. The function returns a reference rather than a copy, so the original data can be accessed and modified directly. Example: int& getRef() { return value; } and then getRef() = 100; modifies the original value.

Q8. What is an inline function?

Ans. A function declared with the inline keyword whose code is copied at the place where it is called. It avoids function call overhead and is best used for small functions.

Q9. How does an inline function differ from a normal function?

Ans. In a normal function the control jumps to the function code and comes back, which takes time. In an inline function the code is expanded at the calling place, so there is no jump and small functions run faster.

Q10. What is a default argument? What is the rule about its position?

Ans. A default argument is a value used when the caller does not provide that argument, for example void add(int a, int b = 5). Default arguments must be the trailing (rightmost) parameters.

Q11. What is a const argument and why is it used?

Ans. It is a parameter declared const so that the function cannot modify it. It protects parameters from unintended modification and is commonly used with references and pointers.

Q12. What is function overloading?

Ans. Function overloading means that two or more functions can have the same name but different parameter lists. The functions may differ in the number, type, or order of parameters.

Q13. On what basis does the compiler select an overloaded function?

Ans. On the function signature, that is the number, type and order of the arguments passed. This selection is made at compile time.

Q14. Can two functions with the same name and the same parameter list but different return types be overloaded?

Ans. No. The return type alone cannot distinguish overloaded functions; the parameter list must differ.

Q15. What is a friend function?

Ans. A friend function is a non-member function declared with the friend keyword inside a class, which is allowed to access the private and protected members of that class.

Q16. Is a friend function a member of the class?

Ans. No. It is declared inside the class but defined outside like an ordinary function, and it is called without using an object.

Q17. Is friendship inherited, and is it mutual?

Ans. No. Friendship is neither inherited by derived classes nor mutual between classes unless it is explicitly declared.

Q18. What is a virtual function?

Ans. A virtual function is a member function in a base class, declared with the virtual keyword, that is expected to be overridden in a derived class. It enables runtime polymorphism.

Q19. What is method overriding?

Ans. Method overriding occurs when a derived class provides its own version of a function that is already defined in its base class, with the same name and the same parameters.

Q20. Differentiate method overloading and method overriding.

Ans. Overloading: same function name with different parameters, may be in the same class, resolved at compile time. Overriding: same function name and same parameters, one in the base and one in the derived class, resolved at runtime when the function is virtual.

Unit 3 : Classes and Objects

Q1. What is a class?

Ans. A class is a user-defined data type that binds data members and member functions together. It serves as a blueprint from which objects are created.

Q2. What is an object?

Ans. An object is an actual instance of a class. Memory is allocated only when an object is created, not when the class is defined.

Q3. Name the three access specifiers in C++ and state their meaning.

Ans. private: members are accessible only within the class, and it is the default for class members. public: members are accessible from anywhere, including outside the class. protected: members are accessible within the class and its derived classes but not from outside.

Q4. How is a member function defined outside a class?

Ans. Using the scope resolution operator ::, for example void Student::display() { }.

Q5. What is nesting of member functions?

Ans. It is a member function calling another member function of the same class directly, without using an object name or the dot operator.

Q6. What is a private member function and how can it be called?

Ans. A private member function is declared in the private section of a class. It cannot be called through an object from outside the class; only other member functions of the same class can call it.

Q7. What is a static data member?

Ans. A static data member is shared by all objects of the class. Only one copy exists for the whole class, it belongs to the class rather than to any object, and it must be defined outside the class.

Q8. How is a static data member defined outside the class?

Ans. By writing the type, class name, scope resolution operator and initial value, for example int Counter::count = 0;

Q9. What is a static member function?

Ans. A static member function is a function that can access only the static members of the class. It can be called using the class name without creating an object.

Q10. Why cannot a static member function access non-static members directly?

Ans. Because it does not work on any particular object, so there is no this pointer available to it to identify which object's members to use.

Q11. What is an array of objects?

Ans. An array whose elements are objects, created in the same way as an array of any other type, for example Item items[3];. Each element is a separate object accessed using an index.

Q12. How can an object be passed to a function?

Ans. It can be passed by value, where a copy of the object is passed, or by reference, where the actual object is passed. Passing by reference is more efficient and allows the function to modify the original object.

Q13. What is a constructor?

Ans. A constructor is a special member function that has the same name as the class, has no return type, and is called automatically when an object is created. It is used to initialize the object.

Q14. Name the types of constructors.

Ans. Default constructor, parameterized constructor, overloaded constructors, constructor with default arguments, and dynamic initialization of objects.

Q15. What is a default constructor?

Ans. A constructor that takes no arguments and is called automatically when an object is created.

Q16. What is constructor overloading?

Ans. Having multiple constructors in the same class with different parameter lists. The appropriate constructor is called automatically based on the arguments passed.

Q17. What is dynamic initialization of objects?

Ans. Initializing objects with values that are supplied at runtime, for example reading the radius from the user and then creating the object Circle c(r);.

Q18. What is a destructor?

Ans. A destructor is a special member function whose name is the class name preceded by a tilde ~, with no arguments and no return type. It is called automatically when an object goes out of scope or is explicitly deleted, and it is used to release resources.

Q19. Why is a destructor important when new is used?

Ans. Because memory allocated with new must be freed with delete. Without a destructor that memory is never freed even after the object is destroyed, which causes a memory leak.

Q20. In what order are the constructor and destructor called for an object created inside a block?

Ans. The constructor is called when the object is created, and the destructor is called automatically when the object goes out of scope at the end of the block.

Q21. What is the string class and how does it differ from a C-style string?

Ans. string is a class provided by the Standard Template Library for handling sequences of characters. Unlike C-style char arrays, string objects grow and shrink dynamically, have built-in functions for manipulation, and avoid buffer overflows.

Q22. Which header file is required for the string class?

Ans. The <string> header, along with using namespace std;.

Q23. How is a full line containing spaces read into a string?

Ans. Using getline(cin, name); which reads the entire line including spaces.

Q24. Name any five string manipulation functions.

Ans. append(), insert(), replace(), erase(), substr() and find() (any five are correct).

Q25. What does find() return when the substring is not found?

Ans. It returns string::npos.

Q26. What is the difference between size() and length()?

Ans. Both return the number of characters in the string and are functionally the same.

Q27. What does capacity() return?

Ans. It returns the total allocated space for the string in characters, which is usually greater than or equal to the actual size so that the string can grow without reallocating memory each time.

Q28. What does resize(n) do?

Ans. It changes the size of the string to n characters, trimming the string if it is shortened or extending it if it is lengthened.

Q29. What is the difference between at() and the [] operator?

Ans. at() performs bounds checking and throws an exception if the index is invalid, whereas the [] operator does not check bounds.

Q30. What does compare() return?

Ans. It compares two strings lexicographically and returns 0 if they are equal, a negative value if the first string is less than the second, and a positive value if it is greater.

Q31. What does swap() do?

Ans. It exchanges the contents of two strings, for example s1.swap(s2);.

Unit 4 : Inheritance

Q1. What is inheritance?

Ans. Inheritance is the OOP feature by which a derived (child) class acquires the data members and member functions of a base (parent) class. It models an IS-A relationship, for example Student IS-A Person.

Q2. Give the syntax of inheritance.

Ans. class Derived : access_mode Base { };

Q3. What are the advantages of inheritance?

Ans. Code reusability by avoiding rewritten common code, extensibility, improved maintenance because common features stay in one place, real-world modelling through IS-A relationships, and support for polymorphism.

Q4. Name the five types of inheritance in C++.

Ans. Single inheritance, multilevel inheritance, multiple inheritance, hierarchical inheritance and hybrid inheritance.

Q5. What is single inheritance?

Ans. A derived class inherits from only one base class: Base to Derived.

Q6. What is multilevel inheritance?

Ans. A class is derived from another derived class, forming a chain of inheritance such as Base to Derived1 to Derived2.

Q7. What is multiple inheritance?

Ans. A class inherits from more than one base class at the same time.

Q8. What is hierarchical inheritance?

Ans. Multiple classes inherit from the same single base class.

Q9. What is hybrid inheritance?

Ans. A combination of two or more types of inheritance, most commonly hierarchical combined with multiple inheritance. Virtual base classes are often used in it to solve ambiguity.

Q10. How do the three inheritance access modes differ?

Ans. In public inheritance, public members of the base stay public and protected members stay protected in the derived class. In protected inheritance, both public and protected members of the base become protected in the derived class. In private inheritance, both become private in the derived class.

Q11. Are the private members of the base class accessible in the derived class?

Ans. No, they are not directly accessible in the derived class. They can be used only through the public or protected member functions of the base class.

Q12. What is the diamond problem?

Ans. When a class is derived from two classes that both inherit from a common base class, the derived class receives two copies of that base class. References to base members then become ambiguous, and this is called the diamond problem.

Q13. How is the diamond problem solved?

Ans. By making the intermediate classes inherit the common base virtually, for example class Student : virtual public Person { };. Then only one shared copy of the base class exists in the derived object.

Q14. What is a virtual base class?

Ans. A base class that is inherited using the virtual keyword so that only a single copy of it exists in the derived object. It is used to resolve ambiguity in multiple inheritance and to save memory.

Q15. What is an abstract class?

Ans. An abstract class is a class that cannot be instantiated, that is, no objects of it can be created. It is created by declaring at least one pure virtual function and is designed to be used as a base class providing a common interface.

Q16. What is a pure virtual function? Give its syntax.

Ans. A pure virtual function is a virtual function with no implementation in the base class. Its syntax is virtual void draw() = 0; where = 0 tells the compiler that it must be overridden in the derived class.

Q17. What happens if a derived class does not override all the pure virtual functions?

Ans. The derived class also becomes abstract and cannot be instantiated.

Q18. Can an abstract class have constructors and normal functions?

Ans. Yes. An abstract class can have constructors, destructors, data members and normal member functions. It becomes abstract only because it has at least one pure virtual function.

Q19. In what order are constructors executed in inheritance?

Ans. The base class constructor is executed first, and then the derived class constructor is executed. This ensures the base part of the object is initialized before the derived part.

Q20. In what order are destructors executed in inheritance?

Ans. The derived class destructor is executed first, followed by the base class destructor, which is the reverse of the order of construction.

Q21. How does a derived class call a parameterized base class constructor?

Ans. Explicitly, using the initializer list of its own constructor, for example Derived(int a, int b) : Base(a) { }.

Q22. In what order are base class constructors called in multiple inheritance?

Ans. In the order of inheritance, from left to right.

Q23. What type of relationship does inheritance represent?

Ans. An IS-A relationship, for example a Student IS-A Person and a Car IS-A Vehicle.

Unit 5 : Polymorphism and Type Conversion

Q1. What is polymorphism?

Ans. Polymorphism means having many forms. In C++ it is the ability of the same function or operator to work differently in different situations. For example, one person can at the same time be a father, a husband and an employee.

Q2. What are the two types of polymorphism in C++?

Ans. Compile-time polymorphism, also called early binding or static polymorphism, and runtime polymorphism, also called late binding or dynamic polymorphism.

Q3. How is compile-time polymorphism achieved?

Ans. By function overloading and operator overloading. The compiler decides at compile time which version to use, based on the context.

Q4. How is runtime polymorphism achieved?

Ans. By function overriding using virtual functions, together with base class pointers or references. The function call is resolved at runtime.

Q5. What is operator overloading?

Ans. Operator overloading means redefining an operator such as +, -, * or / so that it also works with user-defined types such as objects.

Q6. State the rules of operator overloading.

Ans. At least one operand must be a user-defined type such as a class object. The operators ::, ., .*, sizeof and ?: cannot be overloaded.

Q7. What is unary operator overloading? Give an example.

Ans. It is overloading an operator that works on a single operand, such as ++, -- or unary minus. Example: Number operator-() { return Number(-x); } which is called by writing -n1.

Q8. What is binary operator overloading?

Ans. It is overloading an operator that works on two operands, such as +, -, * or /. Example: Complex operator+(Complex c) { return Complex(real + c.real, imag + c.imag); }

Q9. What does the statement c1 + c2 call when operator+ is a member function?

Ans. It calls c1.operator+(c2), that is, the member function is invoked on the left operand with the right operand as the argument.

Q10. How does a friend function differ from a member function for operator overloading?

Ans. A friend function is not a member of the class, so it has no this pointer and both operands are passed to it as arguments. A member function uses this for the left operand and takes only the right operand as an argument.

Q11. What is function overriding?

Ans. Function overriding occurs when a derived class defines its own version of a member function that already exists in the base class, with the same name and the same parameters. The base class function must be virtual for runtime polymorphism to happen.

Q12. What is the difference between static and dynamic binding?

Ans. Static binding is decided at compile time and happens for normal function calls; it is faster but offers no flexibility. Dynamic binding is decided at runtime and happens for virtual functions; it enables polymorphism.

Q13. What is the this pointer?

Ans. The this pointer is a hidden pointer present in every non-static member function that points to the object on which the member function was called.

Q14. Give two uses of the this pointer.

Ans. To disambiguate between parameter names and data member names, as in this->roll = roll;, and to return the current object from a member function.

Q15. How is a pointer to an object declared and used?

Ans. It is declared as Student* ptr = &s1; and member functions are called using the arrow operator, as in ptr->show();.

Q16. What happens when a base class pointer points to a derived class object and the function is not virtual?

Ans. The base class version of the function is executed, because binding is static and is decided at compile

time from the pointer type.

Q17. What happens when the function is virtual?

Ans. The derived class version is executed, because the call is resolved at runtime based on the actual object, which is dynamic binding.

Q18. What is type conversion?

Ans. Type conversion means converting one type of data into another compatible type such that it does not lose its meaning.

Q19. Name the two ways in which type conversion happens in C++.

Ans. Implicit conversion, which is done automatically by the compiler, and explicit or user-defined conversion, which is written by the programmer between basic types and class types.

Q20. How is basic type to class type conversion achieved?

Ans. Through a constructor in the class that takes a single basic-type argument, for example Time(int m) which allows writing Time t = totalMinutes;

Q21. How is class type to basic type conversion achieved?

Ans. Through a conversion function inside the class, for example operator int() { return hours * 60 + minutes; } which allows writing int totalMinutes = t;

Q22. What are the two ways to convert one class type to another?

Ans. By a conversion constructor in the destination class that takes an object of the source class, or by a conversion function in the source class that returns an object of the destination class.

Q23. What does the explicit keyword do?

Ans. It prevents implicit conversions through that constructor and requires the object to be constructed explicitly.

Q24. What is a conversion function and what is its syntax?

Ans. A conversion function is a member function that converts a class object into another type. Its syntax is operator basic_type(); and it has no return type and no parameters.

Unit 6 : Files and Exception Handling

Q1. What is a stream?

Ans. A stream is a flow of data. An input stream carries data from an input device such as the keyboard or a file into the program, and an output stream carries data from the program to an output device such as the screen or a file.

Q2. Name the four standard I/O stream objects and state their purpose.

Ans. cin is the standard input stream of type istream. cout is the standard output stream of type ostream. cerr is the unbuffered standard error stream. clog is the buffered standard error stream used for log messages.

Q3. What is the difference between cerr and clog?

Ans. Both are used for error messages, but cerr is unbuffered so its output appears immediately, while clog is buffered.

Q4. Name the main stream classes and their purpose.

Ans. ios is the base class for all stream classes. istream is the input stream class. ostream is the output stream class. iostream is derived from both and handles input and output. ifstream is the input file stream, ofstream is the output file stream and fstream is a file stream for both input and output.

Q5. Which header file is needed for file handling in C++?

Ans. The <fstream> header file.

Q6. What are the three main steps in file handling?

Ans. Opening the file, performing read or write operations, and closing the file.

Q7. Why must a file be closed after use?

Ans. Closing the file closes the associated stream and frees the resources being used. It is important especially in long running programs to avoid memory leaks and data loss.

Q8. Name the file open modes in C++.

Ans. ios::in opens a file for reading, ios::out opens it for writing, ios::app appends at the end of the file, ios::trunc deletes the old contents, ios::binary opens the file in binary mode, and ios::ate opens the file and moves the pointer to the end.

Q9. How can multiple file open modes be combined?

Ans. Using the bitwise OR operator |, for example ios::in | ios::out or fstream file("data.txt", ios::in | ios::out | ios::app);

Q10. What is the difference between ifstream and ofstream?

Ans. ifstream is the input file stream and is used for reading from files; it is equivalent to opening an fstream in ios::in mode. ofstream is the output file stream used for writing to files; it is equivalent to opening an fstream in ios::out mode.

Q11. What does the fstream class do?

Ans. fstream is a file stream class that supports both input and output operations on the same file.

Q12. How is a file opened using the open() function?

Ans. By calling open() on the stream object with the file name and mode, for example fin.open("file.txt", ios::in);

Q13. What happens if a file opened in read mode does not exist?

Ans. No new file is created and an error or exception occurs because the stream enters a failed state.

Q14. How do you check whether a file was opened successfully?

Ans. By testing the stream object as a condition, for example if (!fout) { cout << "File not created!"; }.

Q15. Which function deletes a file in C++ and which header declares it?

Ans. The remove() function, declared in the <cstdio> (stdio.h) header. It takes the file path as an argument, returns 0 if the file was successfully deleted, and returns a non-zero value if an error occurs.

Q16. Why is binary mode used in file handling?

Ans. In binary mode the file is treated as a sequence of raw bytes and nothing is interpreted or translated. It is more efficient, stores data exactly as it is in memory, takes fewer bytes, and allows arrays, structures and objects to be stored directly.

Q17. What does the statement bout.write((char*)&num, sizeof(num)); do?

Ans. It takes the address of the variable num, reinterprets it as a pointer to a sequence of bytes, and writes sizeof(num) raw bytes from memory directly into the file. The cast is needed because write() expects a const char* pointer while &num is an int*.

Q18. What is the purpose of seekg() and seekp()?

Ans. seekg() moves the get pointer used for input and seekp() moves the put pointer used for output. They allow random access to any position inside a file.

Q19. What is the difference between formatted and unformatted I/O?

Ans. Formatted I/O presents data in human readable form using the << and >> operators. Unformatted I/O reads and writes raw characters without any formatting, so it is faster but less user friendly.

Q20. Name the unformatted input functions.

Ans. get(), getline(), read(), ignore(), peek(), putback() and gcount().

Q21. Name the unformatted output functions.

Ans. put() writes a single character and write() writes a block of binary data.

Q22. What does get() do?

Ans. It reads a single character from the input stream, including whitespace characters such as space and newline.

Q23. What does peek() do?

Ans. It returns the next character in the input stream without removing it, so the character is still available for the next read.

Q24. What does gcount() return?

Ans. It returns the number of characters actually read by the last unformatted input operation.

Q25. What is an exception?

Ans. An exception is an unusual condition or runtime error that occurs during program execution, such as division by zero, accessing invalid memory, or a file I/O failure.

Q26. What are the three categories of errors in programming?

Ans. Syntax errors, which violate language rules and are caught during compilation; logical errors, where the code runs but produces incorrect results; and runtime errors, which occur unexpectedly during execution.

Q27. Which keywords are used for exception handling in C++?

Ans. try, catch and throw.

Q28. What is the purpose of the try block?

Ans. It contains the code that might throw an exception. When an exception occurs inside it, execution stops and control goes to the matching catch block.

Q29. What is the purpose of a catch block?

Ans. It contains the code that handles an exception of a particular type thrown from the try block. The matching catch block is found using the type of the thrown value.

Q30. What are the three types of values that can be thrown as an exception?

Ans. Built-in types such as int, standard exceptions, and custom exceptions written by the programmer.

Q31. Which header defines the standard exception classes?

Ans. The <stdexcept> header. All the standard exception classes are derived from the base class std::exception.

Q32. What is the purpose of the what() method?

Ans. The what() method is present in every standard exception and returns information describing the exception.

Q33. Which standard exception does the vector at() method throw?

Ans. It throws an out_of_range exception when the element with the given index does not exist.

Q34. What is the catch(...) block?

Ans. It is a catch-all block that handles any exception that was not matched by the preceding catch blocks.

Q35. What is the difference between catching by value and catching by reference?

Ans. Catching by value creates a new copy of the thrown object in the catch block. Catching by reference passes a reference to the thrown object instead of creating a copy, and its main advantage is that it allows a base class reference to catch exceptions of derived types.

Q36. Why is catch (exception& e) preferred over catch (exception e)?

Ans. It avoids the overhead of copying the exception object and, more importantly, it can catch any exception derived from exception, since catching a derived exception using a base class reference works.

Q37. What is rethrowing an exception and how is it done?

Ans. Rethrowing means catching an exception inside a try block and, instead of handling it fully, throwing it again so that an outer catch block can handle it. This preserves the type and details of the exception and is done using the bare statement throw; inside the catch block.

Q38. What is noexcept?

Ans. noexcept is an exception specification stating that a function is guaranteed not to throw an exception. noexcept(false) states that the function may throw an exception, which is the default behaviour when no specification is used.

Q39. If you have both catch(runtime_error&) and catch(exception&), which must come first?

Ans. The more derived type, catch(runtime_error&), must come first, otherwise the base class handler catch(exception&) would catch everything and the derived handler would never run.

Q40. Why is exception handling preferred over handling errors with if-else?

Ans. Because it keeps the normal program logic separate from the error handling logic, allows errors to be handled at the appropriate level of the program, and lets the program recover or terminate gracefully instead of terminating abruptly.